Lab 4 Assignment

Student Course Registration System deployed with Docker Compose

Item	Details
Course	Cloud Computing
Instructor	
Student name	Ahmed Salah
Student ID	202200212
Frontend port (Task 7)	8000 + 12 = 8012 -> http://localhost:8012
Submission date	28 October 2026
Deliverables	This report, project folder (course-registration-lab.zip), demo video Lab4_Part2_Demo_AhmedSalah_202200212.mp4 (4 min 54 s)

Summary

The Student Course Registration System runs as five containers started from one compose.yaml: a React frontend served by Nginx, a Node.js/Express API, a MySQL 8.4 database, and two bonus services (a Redis cache and Adminer). Only the frontend is published to the network (port 8012). The database is on its own private network, and its data lives in the named volume mysql-data, so it survives docker compose down / up. All screenshots in this report are real outputs captured on my machine on 28 October 2026.

Tier	Lab (Part 1)	This assignment	Different?
Frontend	HTML/JavaScript + Nginx	React 18 (built with Vite in a multi-stage Dockerfile), served by Nginx	Yes
Backend	Python Flask	Node.js 20 + Express 4 (mysql2, redis client)	Yes
Database	MySQL 8.4	MySQL 8.4 with 3 related tables and foreign keys	Same engine
Bonus	-	Redis 7 cache, Adminer 4.8.1 on 127.0.0.1 only	Added

1. Architecture

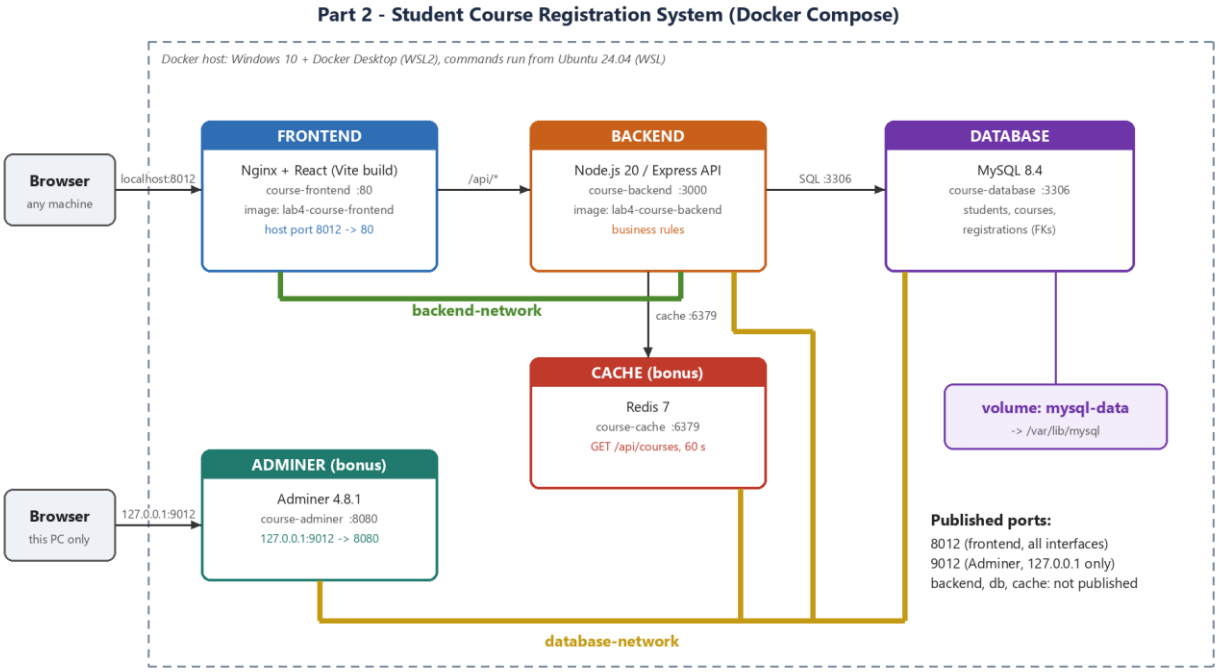


Figure 1: Architecture: containers, networks, volume and published ports (drawn by me)

Container	Image	Networks	Port
course-frontend	lab4-course-frontend (built)	backend-network	8012 -> 80 (published)
course-backend	lab4-course-backend (built)	backend-network, database-network	3000 (internal)
course-database	mysql:8.4	database-network	3306 (internal)
course-cache	redis:7-alpine	database-network	6379 (internal)
course-adminer	adminer:4.8.1	database-network	127.0.0.1:9012 -> 8080

Request path when a user registers a student: Browser -> localhost:8012 (Nginx in course-frontend) -> /api/registrations is proxied to http://backend:3000 -> Express checks the rules inside a MySQL transaction on db:3306 -> clears the cached course list in Redis -> JSON answer back the same way.

2. Task 1 - Environment

Docker Desktop 4 (Docker Engine 29.8.2, Compose v5.6.0) on Windows 10 with the WSL2 back end. All commands were run in an Ubuntu 24.04 (WSL) terminal, so the commands are the same as on an Ubuntu laptop. Docker Desktop's integration lets my user talk to the engine, so sudo is not needed. Because drive C: was almost full, I moved Docker's disk image (docker_data.vhdx) and the project to drive D:.

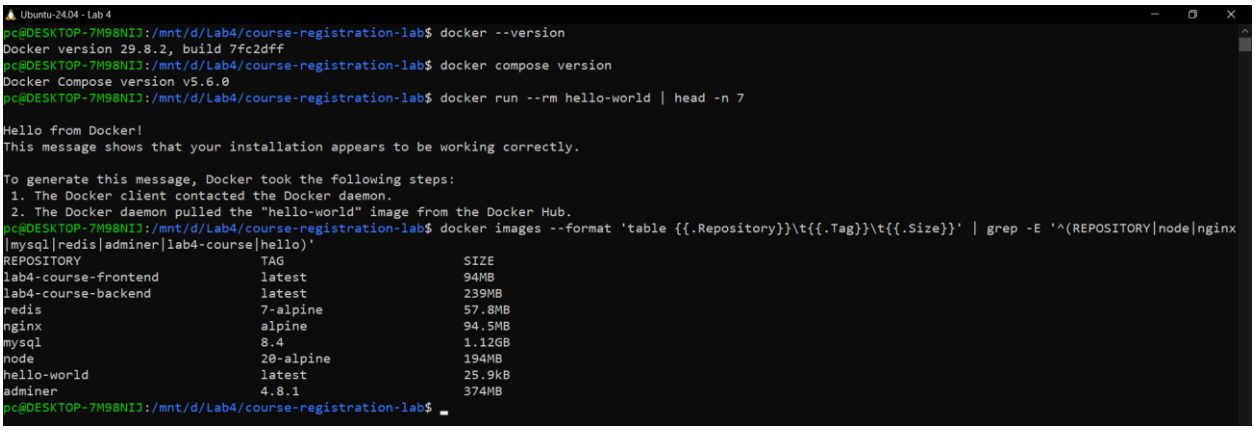


Figure 2: docker --version, docker compose version, hello-world and the pulled images

Image vs container. An image is a read-only, layered template (for example mysql:8.4 or my lab4-course-backend) that is pulled from Docker Hub or built from a Dockerfile. A container is a running instance of an image with its own writable layer, network identity and process. Containers can be deleted and recreated from the same image at any time; anything that must survive goes into a volume.

Images used: node:20-alpine (build base for both custom images), nginx:alpine, mysql:8.4, redis:7-alpine and adminer:4.8.1 come from Docker Hub. lab4-course-frontend and lab4-course-backend are built by me.

3. Task 2 - Build the application

The application provides every feature the assignment asks for:

- **Courses:** add, list (with live seat count) and delete; code, title and number of seats.
- **Students:** add and list (name, email).
- **Registration:** register a student in a course, cancel a registration, list the students of each course.
- **Business rules (enforced in the backend):** no double registration (unique key + 409), a course cannot take more students than its seats (row lock with SELECT ... FOR UPDATE inside a transaction, so two requests cannot take the last seat together), unique emails and course codes (unique keys + 409), input validation (400).
- **Database:** three related tables (students, courses, registrations) with foreign keys (ON DELETE CASCADE) and sample data loaded by db/init.sql on the first start.

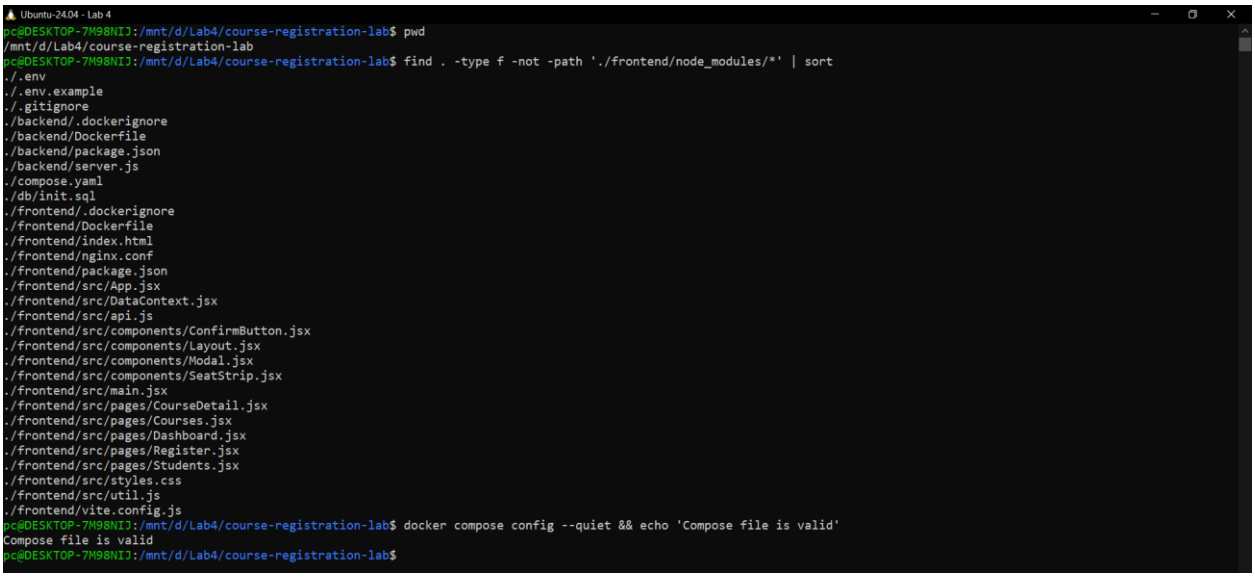


Figure 3: Project tree and docker compose config validation

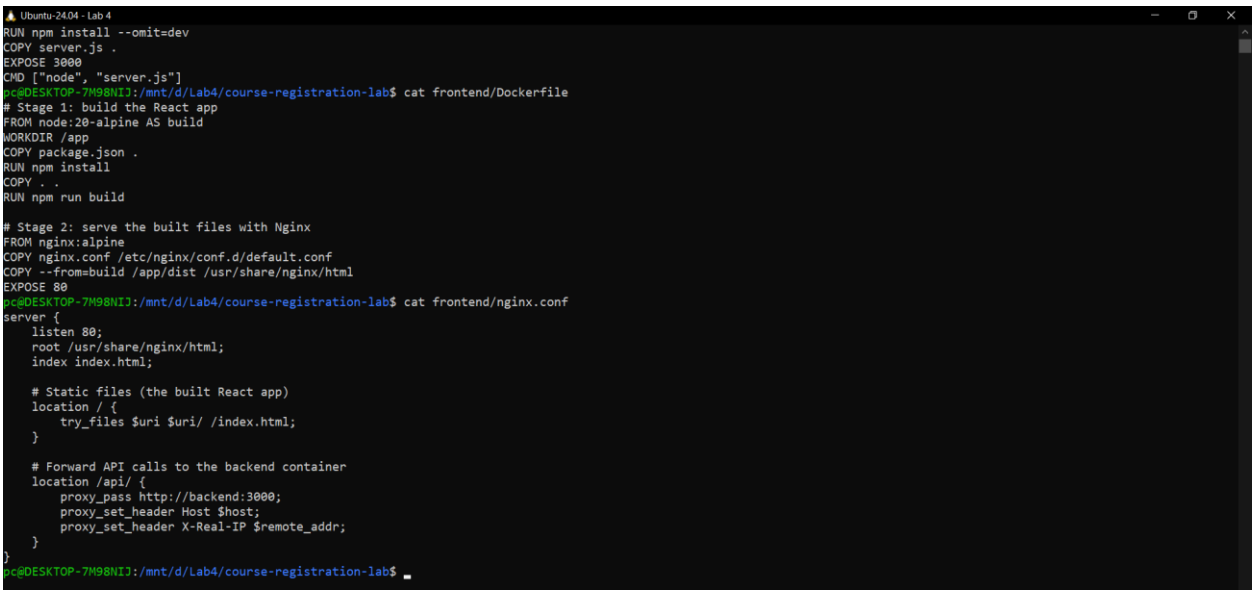


Figure 4: Backend and frontend Dockerfiles and the Nginx configuration

3.1 Project structure

```
course-registration-lab/
|-- compose.yaml          five services, two networks, one volume
```

```
|-- .env                  passwords and ports (not committed, see .env.example)
|-- .env.example
|-- db/init.sql           tables, foreign keys, sample data
|-- backend/              Node.js/Express API
|   |-- Dockerfile
|   |-- package.json
|   `-- server.js
`-- frontend/             React app, built and served by Nginx
    |-- Dockerfile        multi-stage: node build -> nginx
    |-- nginx.conf
    |-- package.json, vite.config.js, index.html
    `-- src/              App, pages (Overview, Courses, CourseDetail, Students, Registrations),
components
```

3.2 Configuration files (passwords removed)

compose.yaml

```
services:
  frontend:
    build: ./frontend
    image: lab4-course-frontend
    container_name: course-frontend
    ports:
      - "${FRONTEND_PORT}:80"
    depends_on:
      - backend
    networks:
      - backend-network
    restart: unless-stopped

  backend:
    build: ./backend
    image: lab4-course-backend
    container_name: course-backend
    environment:
      DB_HOST: db
      REDIS_URL: redis://cache:6379
      MYSQL_DATABASE: ${MYSQL_DATABASE}
      MYSQL_USER: ${MYSQL_USER}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD}
    depends_on:
      db:
        condition: service_healthy
      cache:
        condition: service_healthy
    networks:
      - backend-network
      - database-network
    restart: unless-stopped

  db:
    image: mysql:8.4
    container_name: course-database
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: ${MYSQL_DATABASE}
      MYSQL_USER: ${MYSQL_USER}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD}
    volumes:
      - mysql-data:/var/lib/mysql
      - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    healthcheck:
      test:
        - CMD-SHELL
        - mysqladmin ping -h localhost -uroot -p"$${MYSQL_ROOT_PASSWORD}" --silent
      interval: 10s
```

```
      timeout: 5s
      retries: 10
      start_period: 30s
    networks:
      - database-network
    restart: unless-stopped

  # Bonus: Redis cache for GET /api/courses (only the backend can reach it)
  cache:
    image: redis:7-alpine
    container_name: course-cache
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 3s
      retries: 5
    networks:
      - database-network
    restart: unless-stopped

  # Bonus: Adminer, published on 127.0.0.1 only (not reachable from other machines)
  adminer:
    image: adminer:4.8.1
    container_name: course-adminer
    ports:
      - "127.0.0.1:${ADMINER_PORT}:8080"
    environment:
      ADMINER_DEFAULT_SERVER: db
    depends_on:
      - db
    networks:
      - database-network
    restart: unless-stopped

volumes:
  mysql-data:

networks:
  backend-network:
  database-network:
```

.env.example (the real .env has the same keys)

```
MYSQL_ROOT_PASSWORD=change-me
MYSQL_DATABASE=coursesdb
MYSQL_USER=courseuser
MYSQL_PASSWORD=change-me
FRONTEND_PORT=8012
ADMINER_PORT=9012
```

backend/Dockerfile

```
FROM node:20-alpine
WORKDIR /app
COPY package.json .
RUN npm install --omit=dev
COPY server.js .
EXPOSE 3000
CMD ["node", "server.js"]
```

frontend/Dockerfile

```
# Stage 1: build the React app
FROM node:20-alpine AS build
WORKDIR /app
COPY package.json .
RUN npm install
COPY . .
```

```
RUN npm run build

# Stage 2: serve the built files with Nginx
FROM nginx:alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
```

frontend/nginx.conf

```
server {
    listen 80;
    root /usr/share/nginx/html;
    index index.html;

    # Static files (the built React app)
    location / {
        try_files $uri $uri/ /index.html;
    }

    # Docker's internal DNS. Looking the backend up again every 10 s means a
    # restarted backend (new IP) is found without restarting Nginx, and a
    # stopped backend gives a quick 502 instead of a 60 s timeout.
    resolver 127.0.0.11 valid=10s;

    # Forward API calls to the backend container
    location /api/ {
        set $backend http://backend:3000;
        proxy_pass $backend;
        proxy_connect_timeout 5s;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

db/init.sql

```
CREATE TABLE IF NOT EXISTS students (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL,
  CONSTRAINT uq_email UNIQUE (email)
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS courses (
  id INT AUTO_INCREMENT PRIMARY KEY,
  code VARCHAR(20) NOT NULL,
  title VARCHAR(150) NOT NULL,
  seats INT NOT NULL CHECK (seats > 0),
  CONSTRAINT uq_code UNIQUE (code)
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS registrations (
  id INT AUTO_INCREMENT PRIMARY KEY,
  student_id INT NOT NULL,
  course_id INT NOT NULL,
  CONSTRAINT uq_registration UNIQUE (student_id, course_id),
  CONSTRAINT fk_reg_student FOREIGN KEY (student_id) REFERENCES students(id) ON DELETE CASCADE,
  CONSTRAINT fk_reg_course  FOREIGN KEY (course_id)  REFERENCES courses(id)  ON DELETE CASCADE
) ENGINE=InnoDB;

INSERT INTO courses (code, title, seats) VALUES
  ('CC101', 'Cloud Computing', 2),
  ('NET201', 'Computer Networks', 30);

INSERT INTO students (name, email) VALUES
  ('Sample Student', 'sample@example.com');
```

3.3 Main application code - backend/server.js

backend/server.js

```
const express = require("express");
const mysql = require("mysql2/promise");
const { createClient } = require("redis");

const app = express();
app.use(express.json());

// One log line per request, so `docker compose logs backend` shows the traffic
app.use((req, res, next) => {
  const start = Date.now();
  res.on("finish", () => {
    const cache = res.get("X-Cache") ? " cache=" + res.get("X-Cache") : "";
    console.log(`${req.method} ${req.originalUrl} ${res.statusCode} ${Date.now() - start}ms${cache}`);
  });
  next();
});

// Connection settings come from environment variables (set in compose.yaml)
const pool = mysql.createPool({
  host: process.env.DB_HOST || "db",
  user: process.env.MYSQL_USER,
  password: process.env.MYSQL_PASSWORD,
  database: process.env.MYSQL_DATABASE,
  waitForConnections: true,
  connectionLimit: 10,
});

// ---------- Cache (Redis) ----------
// The course list is cached for 60 seconds and cleared on every change.
// If Redis is down the API keeps working and reads straight from MySQL.
const cache = createClient({ url: process.env.REDIS_URL || "redis://cache:6379", disableOfflineQueue: true });
cache.on("error", (e) => console.error("Redis error:", e.message));
cache.connect().catch((e) => console.error("Redis not available:", e.message));
const COURSES_KEY = "courses:list";

async function cacheGet(key) {
  if (!cache.isReady) return null;
  try { return await cache.get(key); } catch { return null; }
}
async function cacheSet(key, value) {
  if (!cache.isReady) return;
  try { await cache.set(key, value, { EX: 60 }); } catch {}
}
async function clearCourseCache() {
  if (!cache.isReady) return;
  try { await cache.del(COURSES_KEY); } catch {}
}

class HttpError extends Error {
  constructor(status, message) {
    super(message);
    this.status = status;
  }
}
const wrap = (fn) => (req, res, next) => fn(req, res).catch(next);

// ---------- Business logic (the rules) ----------
function cleanCourse(body) {
  const code = String(body.code || "").trim().toUpperCase();
  const title = String(body.title || "").trim();
  const seats = Number(body.seats);
  if (!code || code.length > 20) throw new HttpError(400, "Course code must be 1 to 20 characters");
  if (!title || title.length > 150) throw new HttpError(400, "Course title must be 1 to 150 characters");
  if (!Number.isInteger(seats) || seats < 1) throw new HttpError(400, "Seats must be a whole number of at least
1");
  return { code, title, seats };
}
```

```
function cleanStudent(body) {
  const name = String(body.name || "").trim();
  const email = String(body.email || "").trim().toLowerCase();
  if (!name || name.length > 100) throw new HttpError(400, "Student name must be 1 to 100 characters");
  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email) || email.length > 150)
    throw new HttpError(400, "Invalid email address");
  return { name, email };
}

// ---------- API ----------
app.get("/api/health", async (req, res) => {
  const cacheState = cache.isReady ? "connected" : "unavailable";
  try {
    await pool.query("SELECT 1");
    res.json({ api: "running", database: "connected", cache: cacheState });
  } catch (e) {
    res.status(503).json({ api: "running", database: "unavailable", cache: cacheState });
  }
});

// Courses
app.get("/api/courses", wrap(async (req, res) => {
  const cached = await cacheGet(COURSES_KEY);
  if (cached) {
    res.set("X-Cache", "HIT").type("application/json").send(cached);
    return;
  }
  const [rows] = await pool.query(`
    SELECT c.id, c.code, c.title, c.seats, COUNT(r.id) AS registered
    FROM courses c LEFT JOIN registrations r ON r.course_id = c.id
    GROUP BY c.id ORDER BY c.id`);
  const body = JSON.stringify(rows);
  await cacheSet(COURSES_KEY, body);
  res.set("X-Cache", "MISS").type("application/json").send(body);
}));

app.post("/api/courses", wrap(async (req, res) => {
  const { code, title, seats } = cleanCourse(req.body || {});
  const [result] = await pool.query(
    "INSERT INTO courses (code, title, seats) VALUES (?, ?, ?)", [code, title, seats]);
  await clearCourseCache();
  res.status(201).json({ id: result.insertId, code, title, seats });
}));

app.delete("/api/courses/:id", wrap(async (req, res) => {
  const [result] = await pool.query("DELETE FROM courses WHERE id = ?", [req.params.id]);
  if (result.affectedRows === 0) throw new HttpError(404, "Course not found");
  await clearCourseCache();
  res.json({ message: "Course deleted" });
}));

// Students of one course
app.get("/api/courses/:id/students", wrap(async (req, res) => {
  const [rows] = await pool.query(`
    SELECT s.id, s.name, s.email
    FROM registrations r JOIN students s ON s.id = r.student_id
    WHERE r.course_id = ? ORDER BY s.name`, [req.params.id]);
  res.json(rows);
}));

// Students
app.get("/api/students", wrap(async (req, res) => {
  const [rows] = await pool.query("SELECT id, name, email FROM students ORDER BY id");
  res.json(rows);
}));

app.post("/api/students", wrap(async (req, res) => {
  const { name, email } = cleanStudent(req.body || {});
  const [result] = await pool.query(
    "INSERT INTO students (name, email) VALUES (?, ?)", [name, email]);
  res.status(201).json({ id: result.insertId, name, email });
}));
```

```
// Registrations
app.get("/api/registrations", wrap(async (req, res) => {
  const [rows] = await pool.query(`
    SELECT r.course_id, r.student_id, s.name, s.email
    FROM registrations r JOIN students s ON s.id = r.student_id
    ORDER BY r.course_id, s.name`);
  res.json(rows);
}));

app.post("/api/registrations", wrap(async (req, res) => {
  const studentId = Number((req.body || {}).student_id);
  const courseId = Number((req.body || {}).course_id);
  if (!Number.isInteger(studentId) || !Number.isInteger(courseId))
    throw new HttpError(400, "student_id and course_id are required");

  const conn = await pool.getConnection();
  try {
    await conn.beginTransaction();
    // Lock the course row so two requests cannot take the last seat together
    const [courses] = await conn.query("SELECT id, seats FROM courses WHERE id = ? FOR UPDATE", [courseId]);
    if (courses.length === 0) throw new HttpError(404, "Course not found");
    const [students] = await conn.query("SELECT id FROM students WHERE id = ?", [studentId]);
    if (students.length === 0) throw new HttpError(404, "Student not found");

    const [[{ taken }]] = await conn.query(
      "SELECT COUNT(*) AS taken FROM registrations WHERE course_id = ?", [courseId]);
    if (taken >= courses[0].seats) throw new HttpError(409, "Course is full");

    await conn.query("INSERT INTO registrations (student_id, course_id) VALUES (?, ?)", [studentId, courseId]);
    await conn.commit();
    await clearCourseCache();
    res.status(201).json({ student_id: studentId, course_id: courseId });
  } catch (e) {
    await conn.rollback();
    throw e;
  } finally {
    conn.release();
  }
}));

app.delete("/api/courses/:courseId/students/:studentId", wrap(async (req, res) => {
  const [result] = await pool.query(
    "DELETE FROM registrations WHERE course_id = ? AND student_id = ?",
    [req.params.courseId, req.params.studentId]);
  if (result.affectedRows === 0) throw new HttpError(404, "Registration not found");
  await clearCourseCache();
  res.json({ message: "Registration cancelled" });
}));

// ---------- Error handling ----------
app.use((err, req, res, next) => {
  if (err.status) return res.status(err.status).json({ error: err.message });
  if (err.code === "ER_DUP_ENTRY") {
    const m = err.sqlMessage || "";
    if (m.includes("uq_registration")) return res.status(409).json({ error: "Student is already registered in this
course" });
    if (m.includes("uq_email")) return res.status(409).json({ error: "Email already exists" });
    if (m.includes("uq_code")) return res.status(409).json({ error: "Course code already exists" });
    return res.status(409).json({ error: "Duplicate value" });
  }
  console.error(err);
  res.status(500).json({ error: "Internal server error" });
});

app.listen(3000, "0.0.0.0", () => console.log("API listening on port 3000"));
```

4. Task 3 - Deploy with Compose

- One compose.yaml with the three required services (frontend, backend, db) plus two bonus services.
- All passwords and the ports are read from .env (variable substitution); nothing secret is in the code.
- MySQL has a healthcheck (mysqladmin ping); Redis has one too (redis-cli ping). The backend uses depends_on with condition: service_healthy, so it starts only when both are ready.
- restart: unless-stopped on every service.

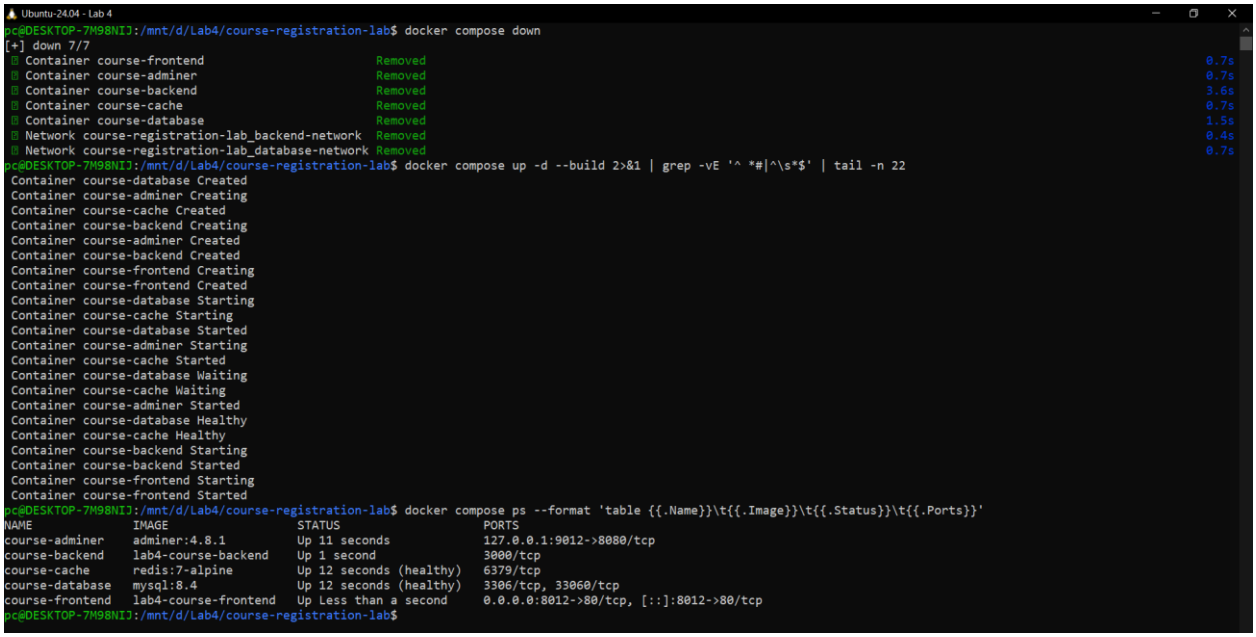


Figure 5: docker compose up -d --build: db and cache become healthy first, then backend and frontend start

5. The running application

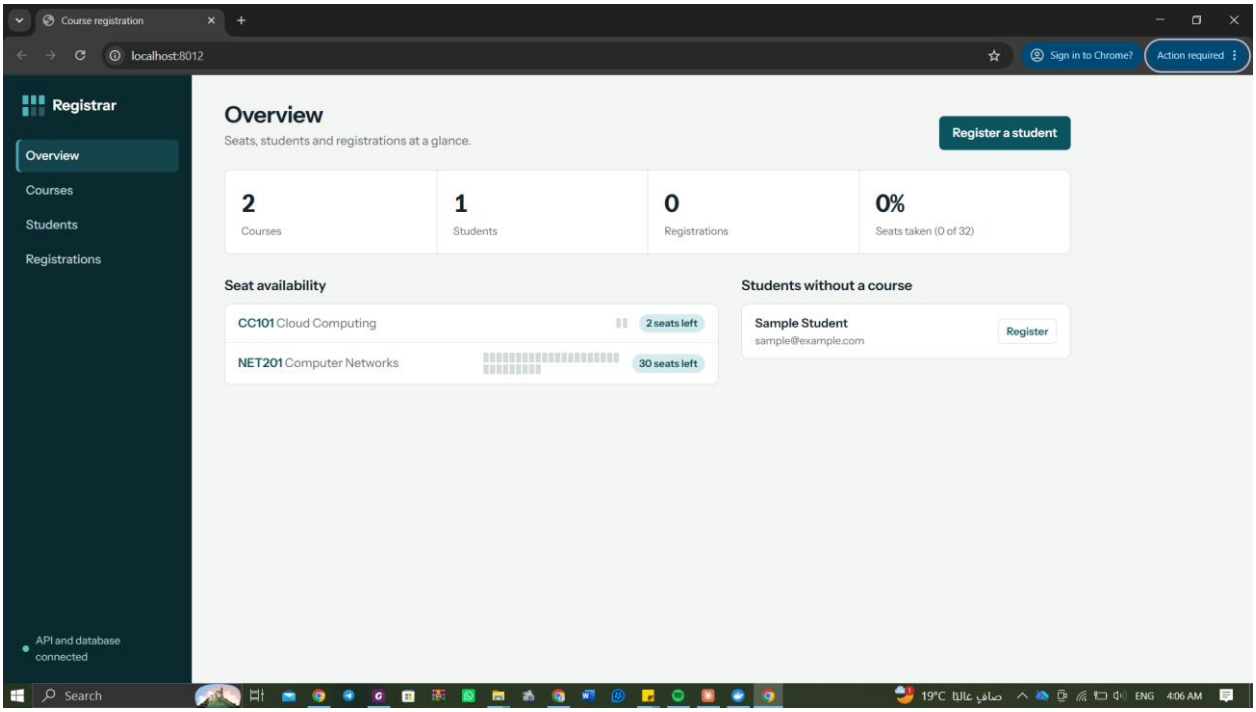


Figure 6: Overview page right after the first start (sample data from init.sql), port 8012 in the address bar

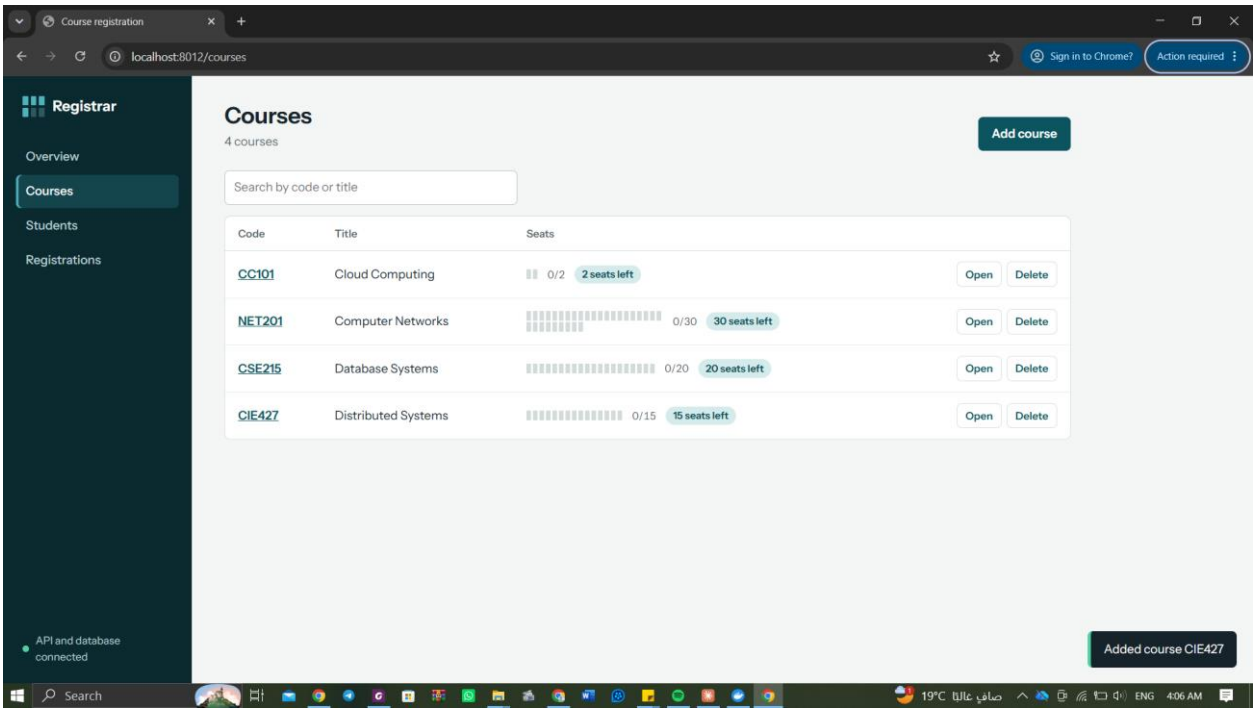


Figure 7: Courses page after adding CSE215 and CIE427 through the UI (4 courses)

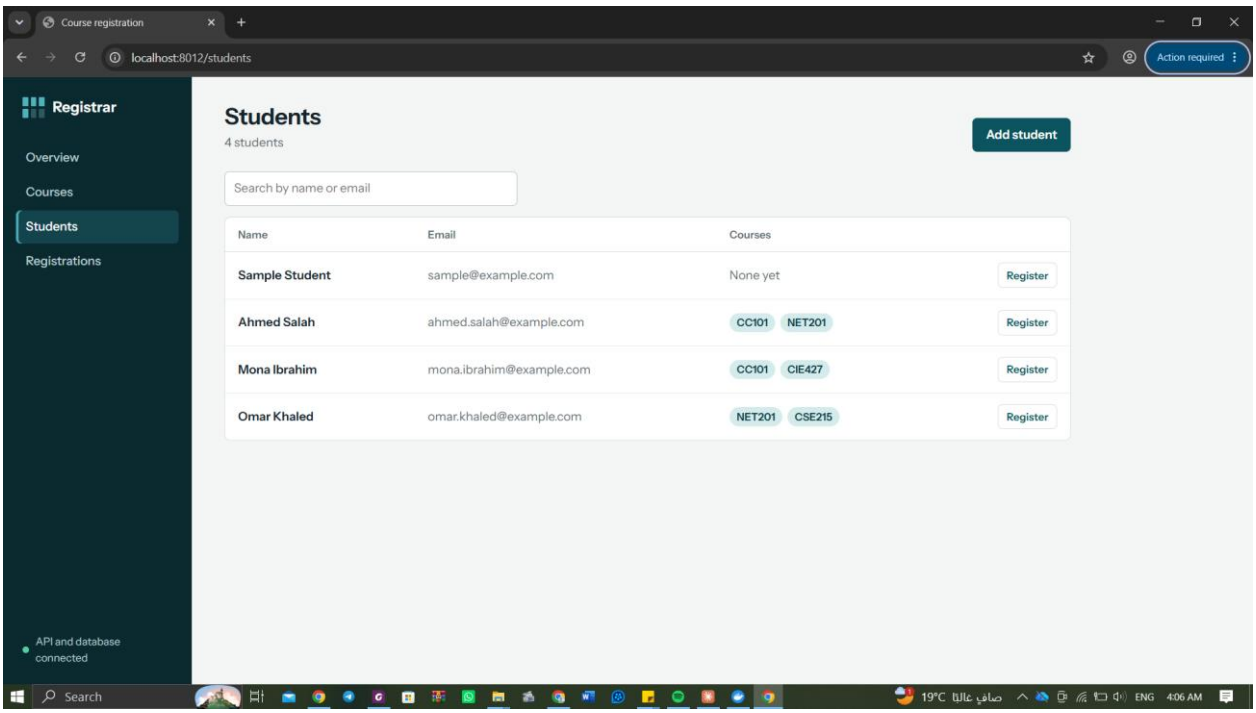


Figure 8: Students page: 4 students and the courses each one is registered in

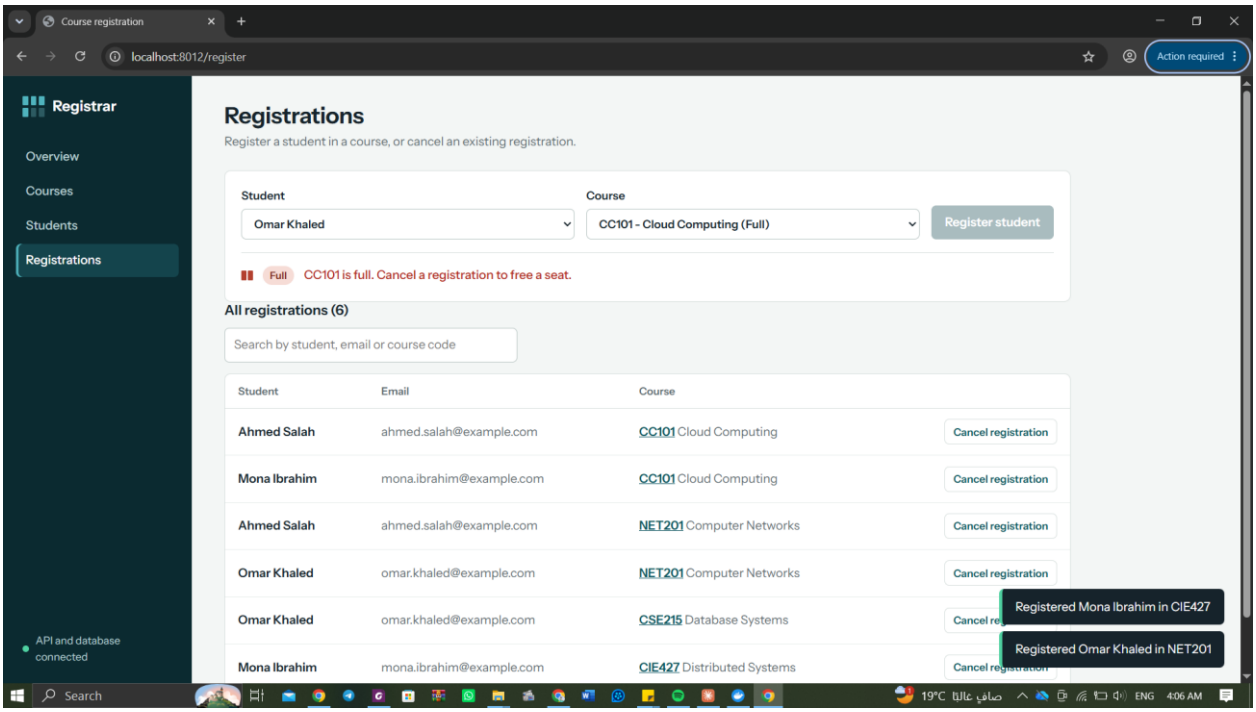


Figure 9: Registrations page: CC101 has 2 seats and is full, so registering Omar Khaled is blocked

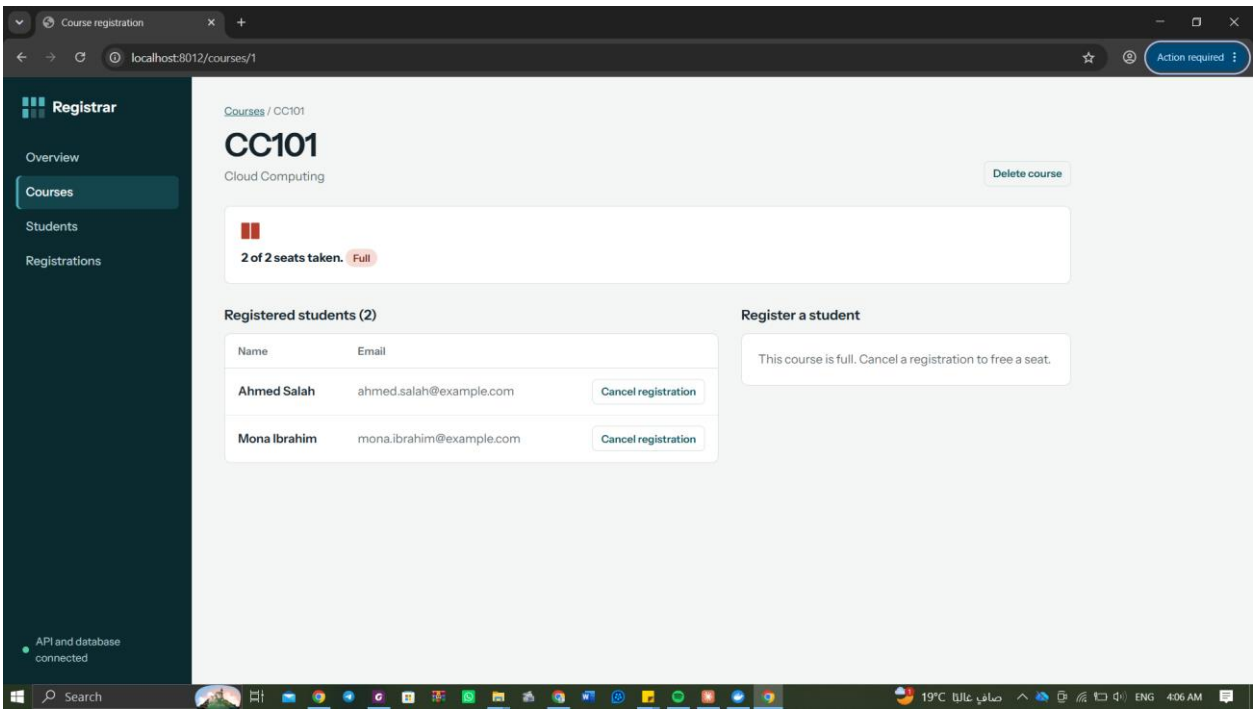


Figure 10: Course detail: the students of CC101, with Cancel registration buttons

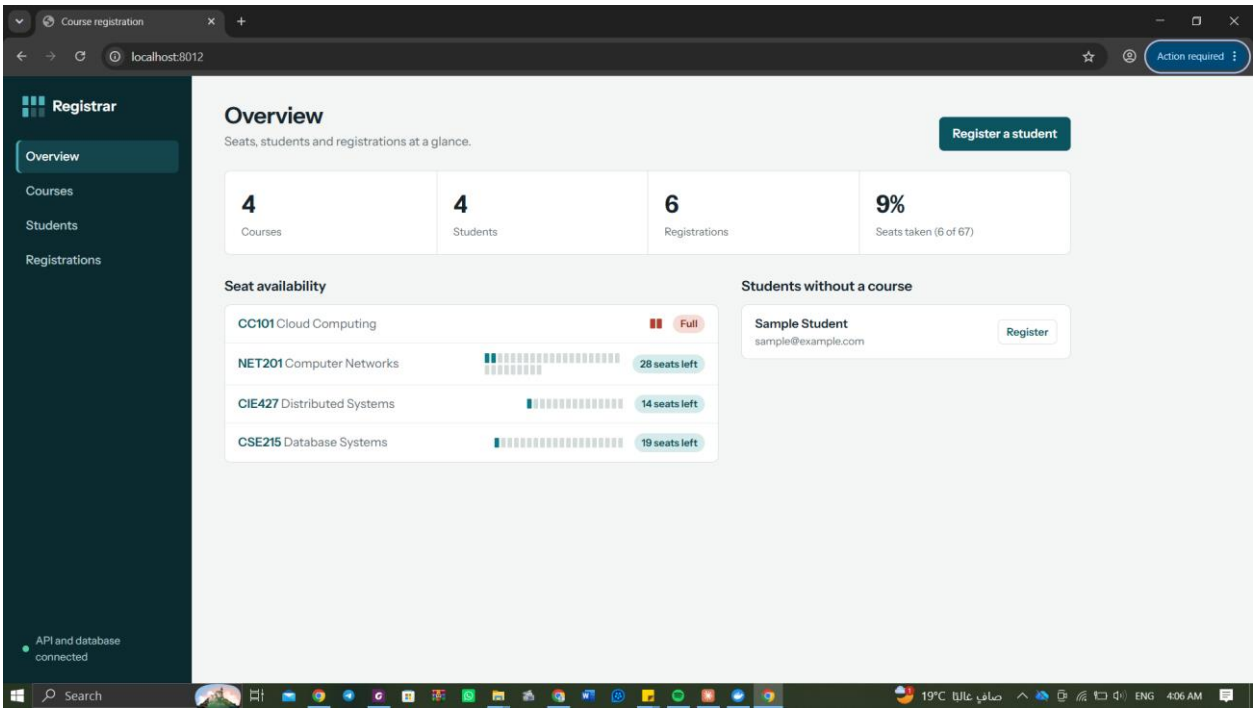


Figure 11: Overview: 4 courses, 4 students, 6 registrations

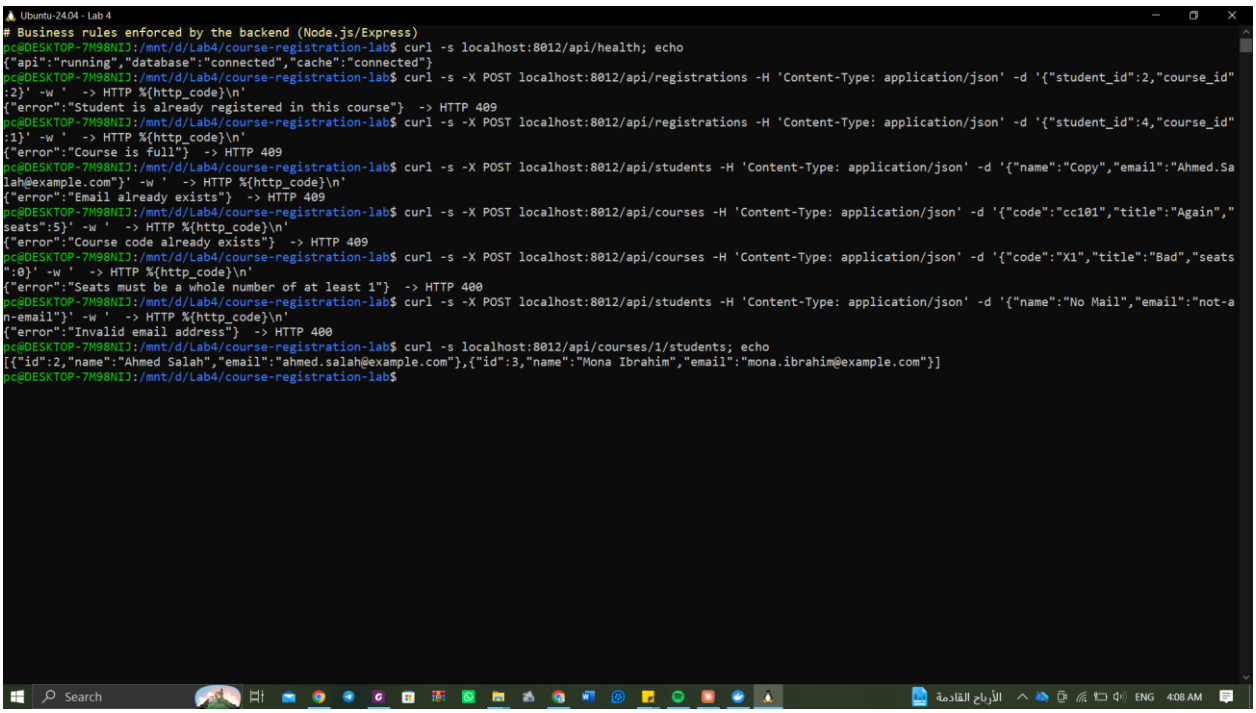


Figure 12: Business rules tested directly against the API: 409 duplicate registration, 409 course full, 409 duplicate email/code, 400 invalid input

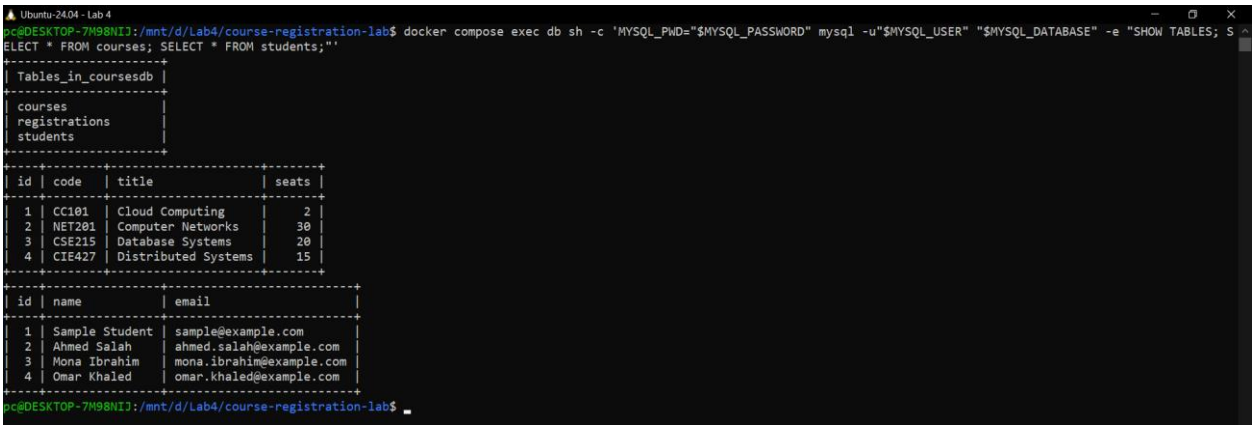


Figure 13: The same data inside MySQL (docker compose exec db mysql ...)

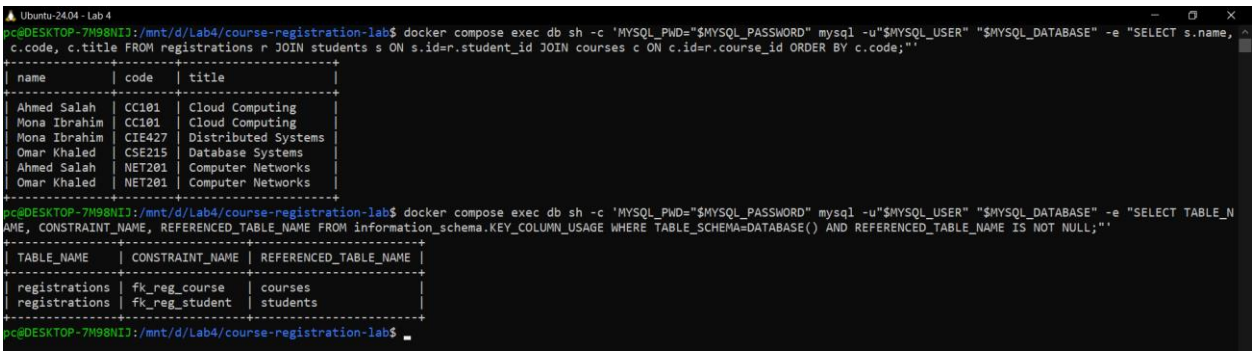


Figure 14: Registrations joined with students and courses, and the two foreign keys

6. Task 4 - Networking

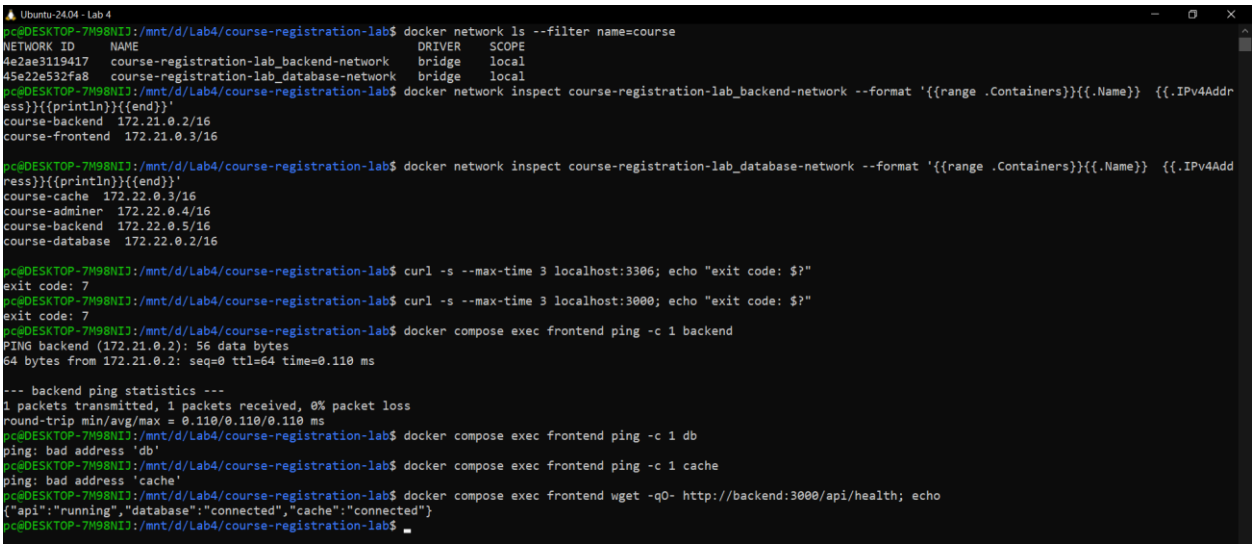


Figure 15: Network inspection: who is on which network, and what can reach what

Two user-defined bridge networks separate the tiers:

- **backend-network:** course-frontend and course-backend.
- **database-network:** course-backend, course-database, course-cache and course-adminer.

How each tier finds the next one. Docker runs an internal DNS server (127.0.0.11) on every user-defined network, and each service is registered under its Compose service name. The browser only knows localhost:8012; port mapping 8012:80 forwards it to Nginx in course-frontend. Nginx serves the built React files and forwards every /api/ request to http://backend:3000; 'backend' is resolved by Docker DNS to the backend container's IP on backend-network. The backend reads DB_HOST=db and REDIS_URL=redis://cache:6379 from its environment, and 'db' and 'cache' resolve on database-network. The backend is the only container on both networks, so it is the only bridge between the web tier and the data tier.

Isolation evidence (Figure above): ping from the frontend to backend works, but ping to db and to cache fails with 'bad address', because the frontend is not on database-network and so the name does not even resolve. From the host, ports 3306 and 3000 are refused (curl exit code 7) because they are not published; only 8012 (and Adminer on 127.0.0.1:9012) are. A user can therefore never reach MySQL directly; all access goes through the API and its business rules.

7. Task 5 - Persistence

Data: 4 courses (CC101, NET201, CSE215, CIE427), 4 students (Ahmed Salah, Mona Ibrahim, Omar Khaled and the sample student) and 6 registrations. Then docker compose down removed all containers and both networks, docker compose ps showed nothing, and docker compose up -d recreated everything.

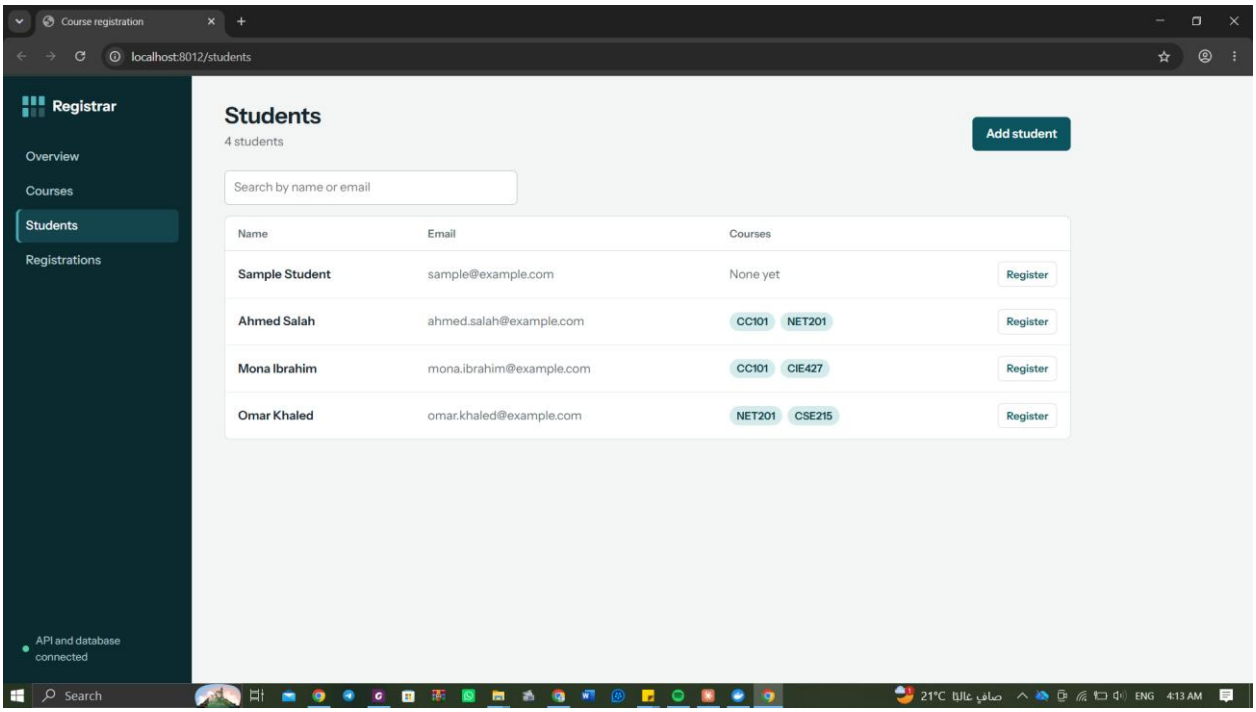


Figure 16: BEFORE: students and their courses in the browser

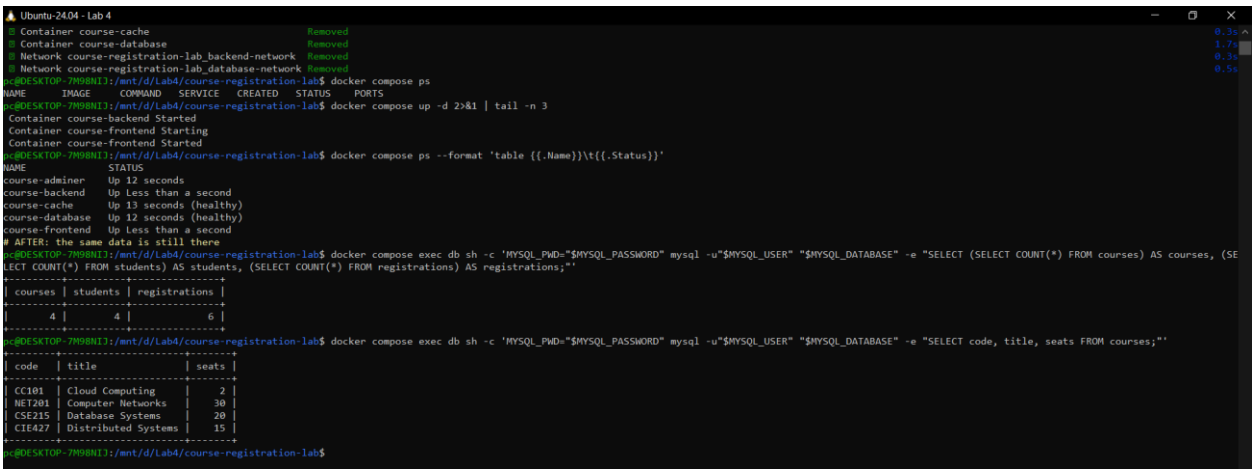


Figure 17: down -> ps (empty) -> up -d -> the same counts (4 / 4 / 6) and courses in MySQL

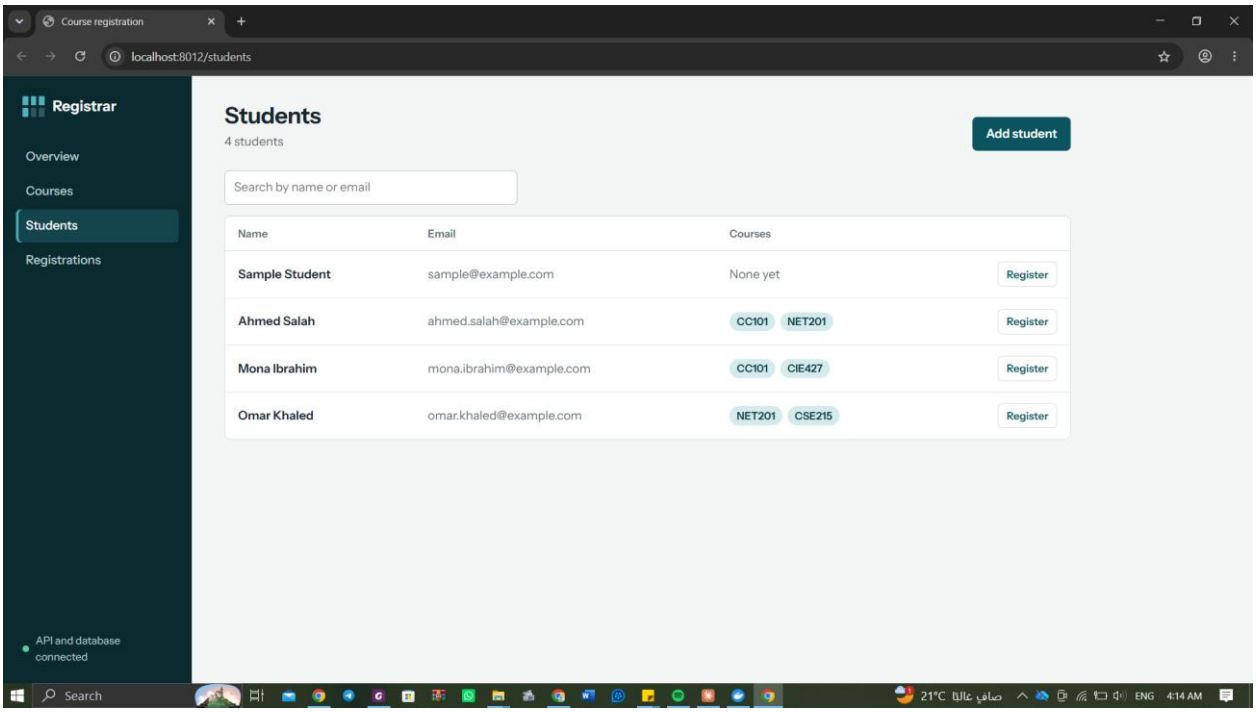


Figure 18: AFTER: the page shows exactly the same data after the containers were recreated

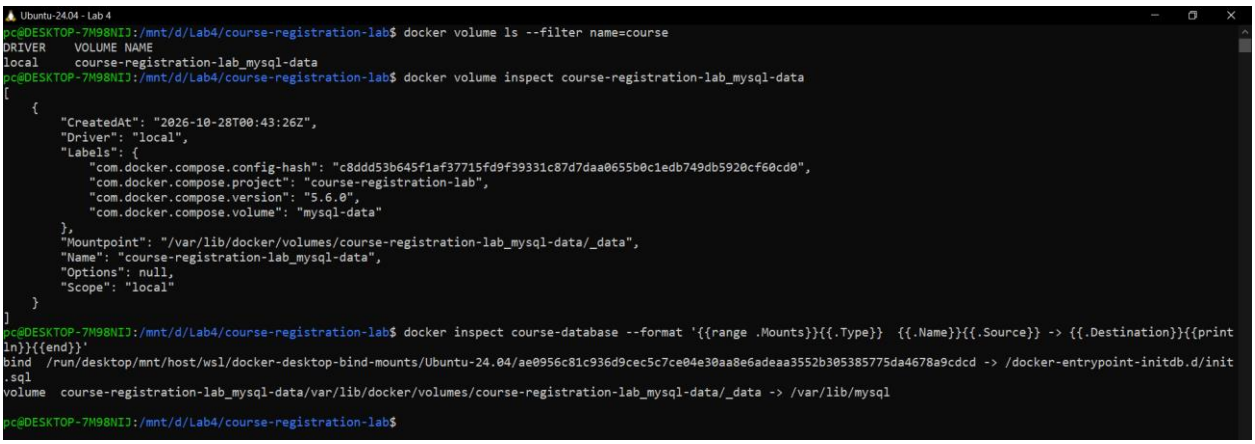


Figure 19: The named volume course-registration-lab_mysql-data mounted at /var/lib/mysql

Why the data survived: the line '- mysql-data:/var/lib/mysql' in the db service mounts a named volume on MySQL's data directory. The volume is managed by Docker and lives outside the container's writable layer, so removing the container does not touch it; the new course-database container mounts the same volume and finds the existing database files. init.sql is not run again because the data folder is not empty. docker compose down -v would also delete the volume and the database would restart with only the sample data.

8. Task 6 - Monitoring and troubleshooting

Monitoring. docker compose ps shows the state and health of every service; docker compose logs shows each service's output. I added a small request logger to the backend (method, URL, status, time and cache HIT/MISS), so the backend log shows the traffic. docker stats shows CPU and memory per container.

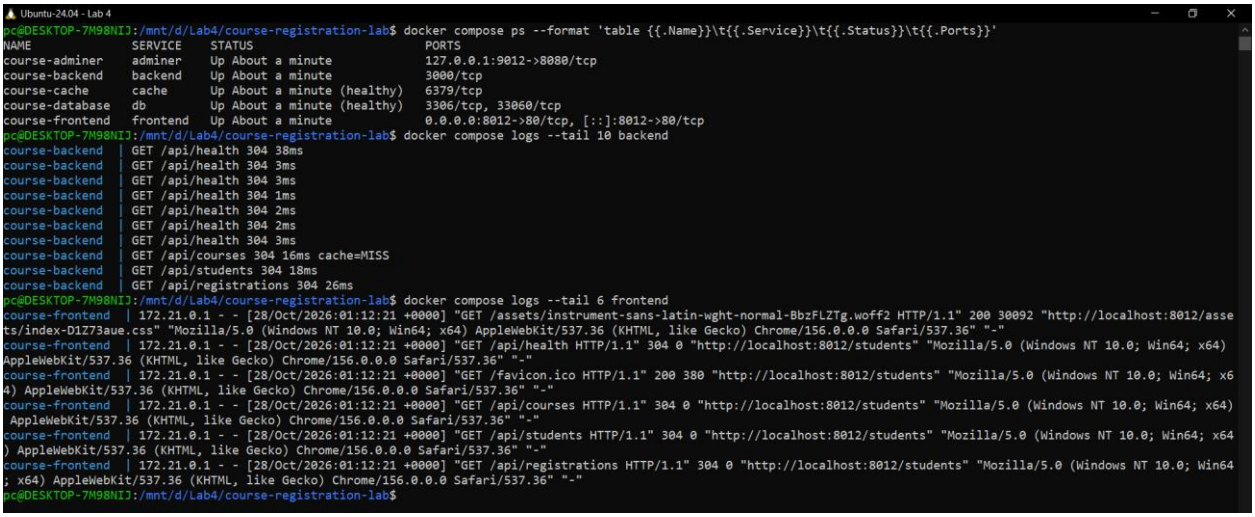


Figure 20: docker compose ps and the logs of the backend (request log) and the frontend (Nginx access log)

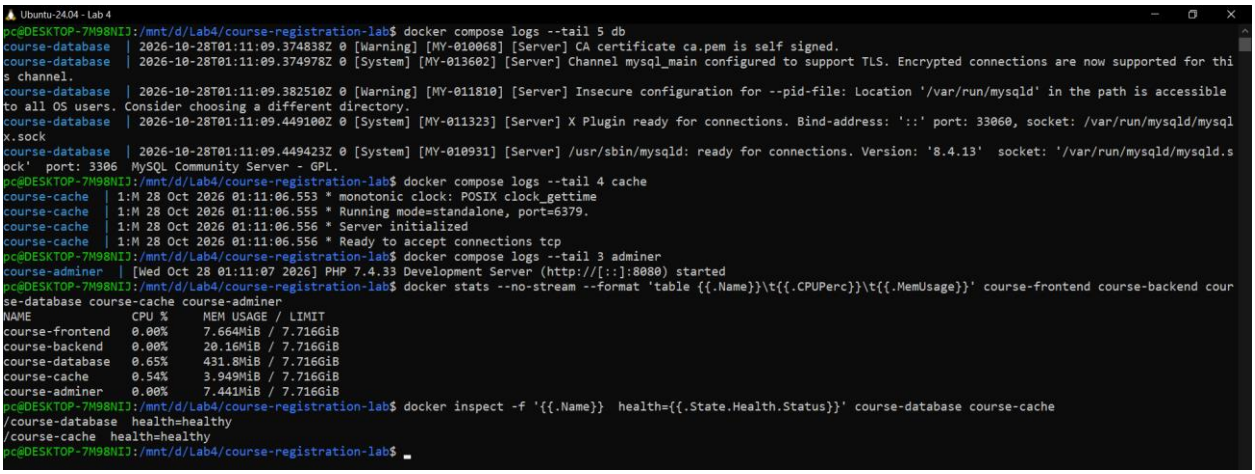


Figure 21: Logs of db, cache and adminer, docker stats, and the health status of db and cache

8.1 Experiment: stopping the backend on purpose

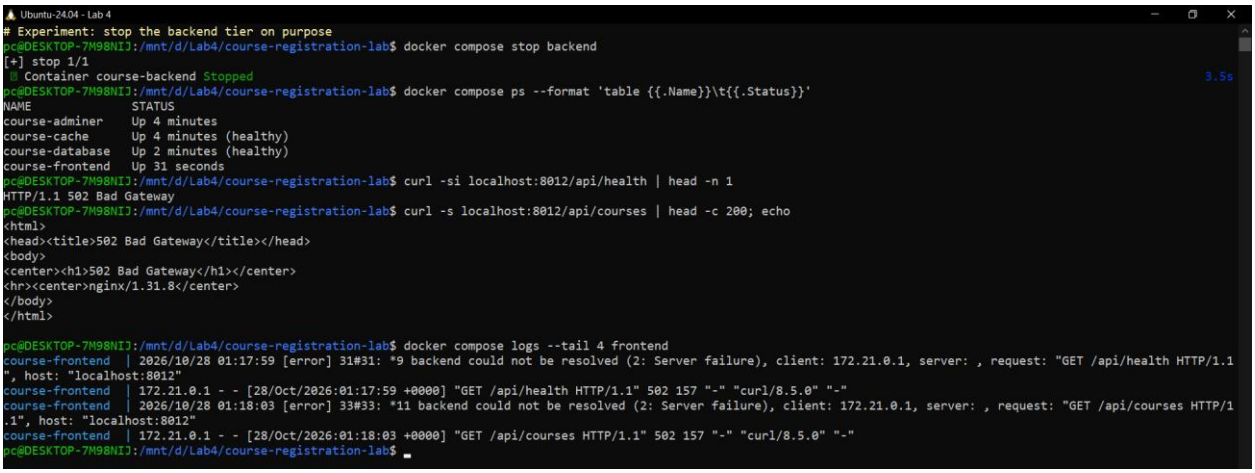


Figure 22: Backend stopped: compose ps no longer lists it, Nginx answers 502 Bad Gateway, the frontend log shows 'backend could not be resolved'

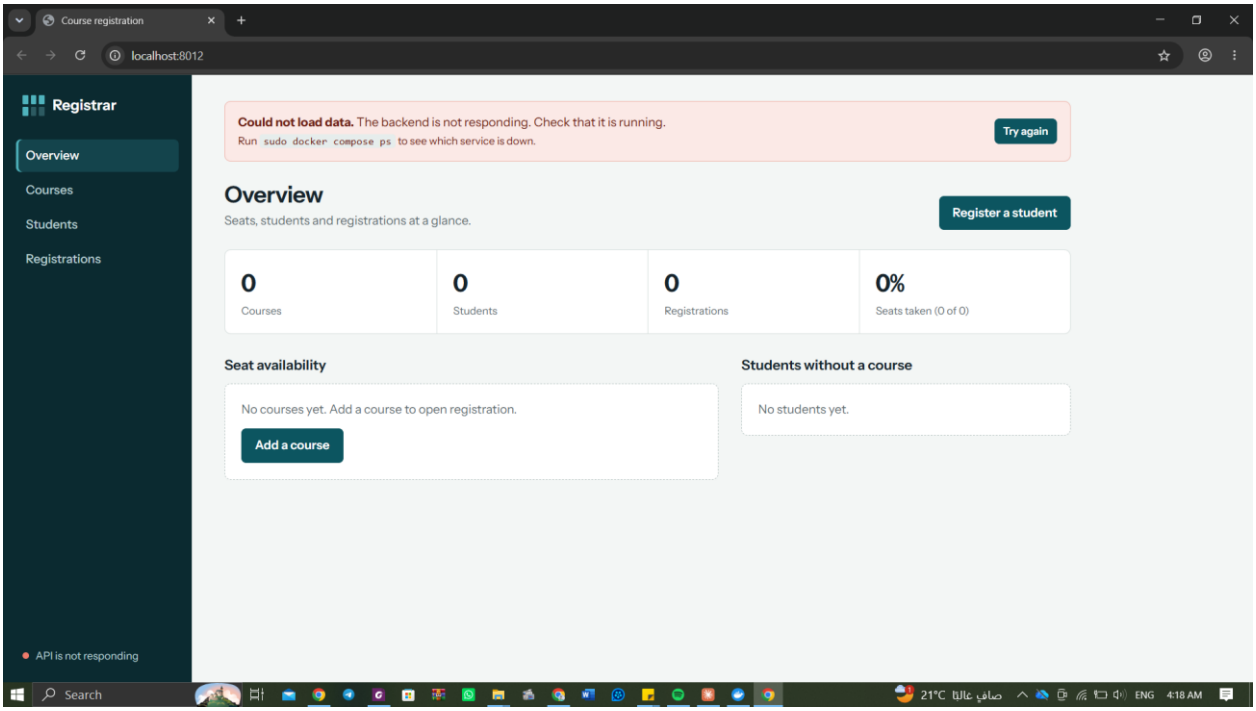


Figure 23: What the user sees: an error banner and the red status 'API is not responding'

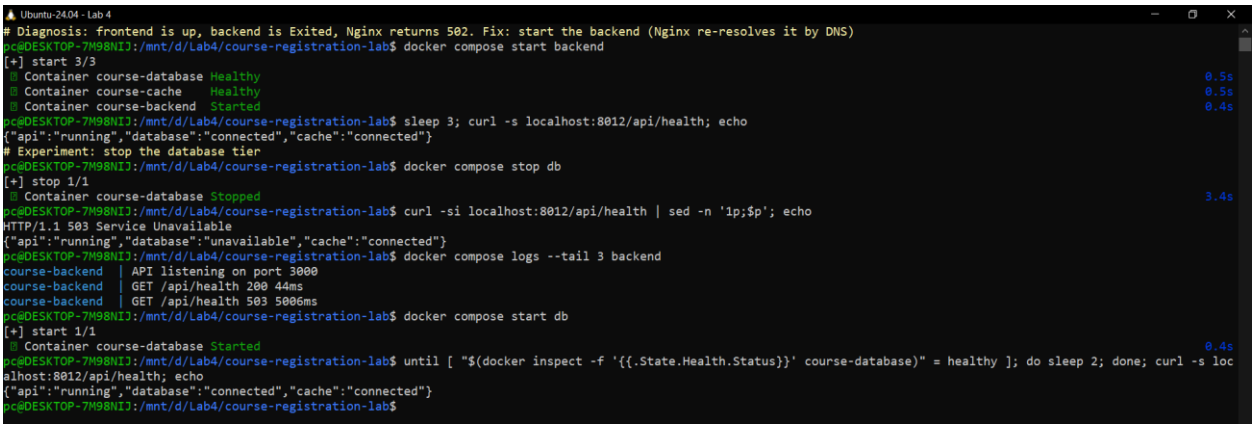


Figure 24: Fix: start the backend (health OK again). Second experiment: stopping db gives 503 'database unavailable', starting it fixes it

Symptoms: the web page still loads (the frontend tier is fine) but shows 'Could not load data. The backend is not responding', the status in the sidebar turns red and every /api call returns 502.

Diagnosis, one tier at a time: (1) docker compose ps - frontend, db and cache are up, backend is missing (Exited). (2) curl -i localhost:8012/api/health returns 502, which means Nginx is up but cannot reach its upstream. (3) docker compose logs frontend shows 'backend could not be resolved', so the backend container is not on the network. Fix: docker compose start backend; /api/health returns {"api":"running","database":"connected","cache":"connected"} and the page recovers with 'Try again'.

Stopping the database instead gives a different symptom: Nginx and the API still answer, but /api/health returns HTTP 503 with "database":"unavailable" and the backend log shows 'GET /api/health 503'. docker compose start db and waiting for 'healthy' fixes it.

8.2 Problems I met and how I solved them

- **Slow failure / stale backend address in Nginx.** With the original 'proxy_pass http://backend:3000;' Nginx resolves the name only once at start-up. When the backend was stopped, requests hung for about 60 s (connect timeout to the old IP) before failing, and after a backend rebuild Nginx could keep the old address (the 502 problem in the lab's troubleshooting table). I added 'resolver 127.0.0.11 valid=10s', a variable in proxy_pass so the name is looked up again, and 'proxy_connect_timeout 5s'. Now a stopped backend gives a fast 502 and a restarted backend is found without restarting the frontend.
- **Disk full.** Drive C: had about 100 MB free and Docker's disk image was on C:, so builds could fail. I moved the Docker Desktop disk image to D:\DockerDesktopWSL and kept the project on D:.
- **Port 8080 in use.** Another program already listens on 8080 on this PC (the Part 1 lab therefore uses 8081). The assignment uses its own port 8012, taken from FRONTEND_PORT in .env.

9. Task 7 - Individual change

Student ID 202200212 ends in 12, so the frontend is published on 8000 + 12 = 8012. The value lives in .env (FRONTEND_PORT=8012) and compose.yaml maps "${FRONTEND_PORT}:80". Evidence: the PORTS column in Figure 5 (0.0.0.0:8012->80/tcp) and the address bar localhost:8012 in every browser screenshot.

10. Bonus

10.1 Cache - Redis (3 marks)

A redis:7-alpine container (course-cache) on database-network caches the response of GET /api/courses for 60 seconds. Every change that affects the course list (add or delete a course, register or cancel) deletes the key. The response header X-Cache shows MISS (read from MySQL) or HIT (read from Redis). If Redis is down the API keeps working and simply reads from MySQL.

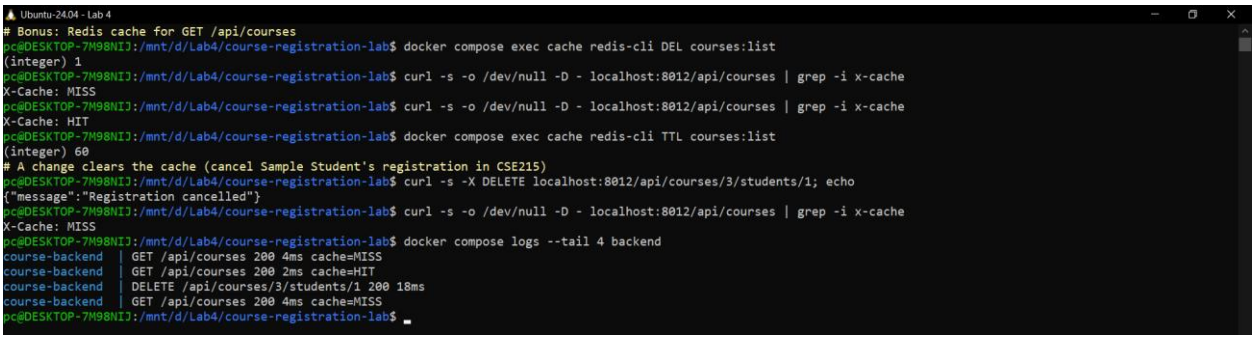


Figure 25: MISS then HIT (23 ms vs 2 ms in the backend log), TTL 60 s, and a cancellation clears the cache (MISS again)

10.2 Admin tool - Adminer (3 marks)

adminer:4.8.1 runs as a fourth service on database-network and is published as "127.0.0.1:${ADMINER_PORT}:8080", so it listens only on the loopback interface of this PC and cannot be reached from other machines (netstat shows 127.0.0.1:9012, while the frontend listens on 0.0.0.0:8012).



Figure 26: Adminer is bound to 127.0.0.1:9012 only

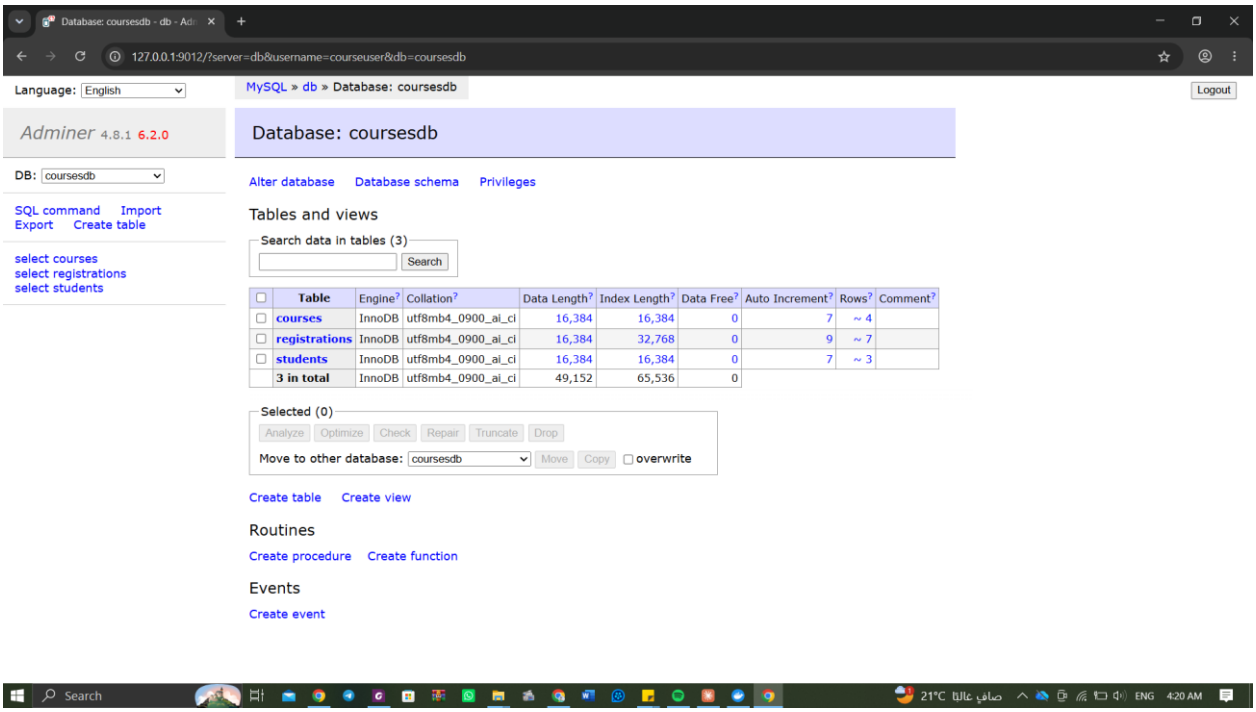


Figure 27: Adminer at http://127.0.0.1:9012 showing the three tables

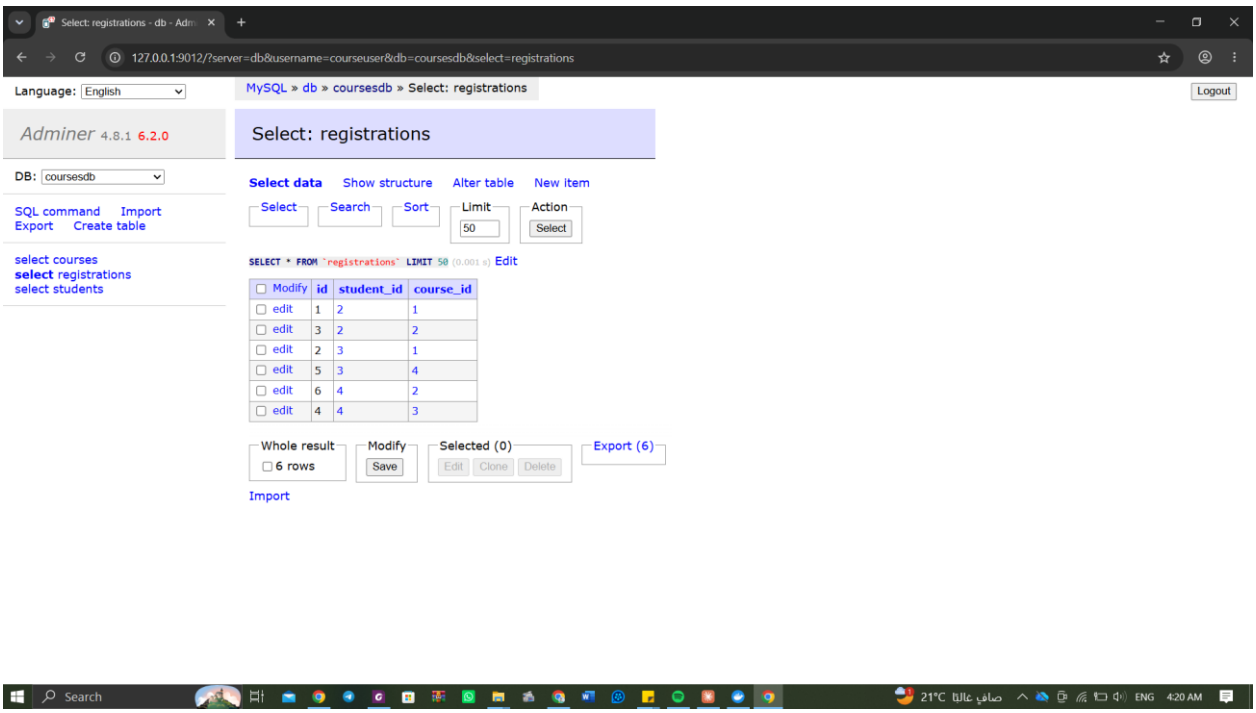


Figure 28: Adminer: contents of the registrations table

10.3 Cloud (AWS EC2)

Not done: it needs an AWS account. The same compose project would run unchanged on an Ubuntu EC2 instance after installing Docker; only port 8012 would be opened in the security group.

11. Live demo video

File: Lab4_Part2_Demo_AhmedSalah_202200212.mp4 (4 min 54 s, 1920x1080), recorded on the real desktop:

Time	What is shown
0:00	Project, services, docker compose down, docker compose up -d --build, docker compose ps (all healthy)
0:50	Using the app at localhost:8012: add course CIE350 (3 seats), add student Youssef Adel, duplicate email rejected, register two students in CIE350, full-course and already-registered rules
2:00	Data in MySQL (courses, students, registrations join) and the Redis cache header
2:30	Stop the backend: page shows the error banner, curl gives 502, frontend log; start it again and the page recovers
3:30	Persistence: counts, docker compose down, up -d, same counts and same data in the browser

12. Deliverables checklist

Deliverable	Where
1. Architecture diagram	Section 1, architecture_diagram.png
2. Project structure, compose.yaml, Dockerfiles, main code	Section 3, course-registration-lab.zip
3. Screenshots of the running app and Evidence column	Sections 2-10, screenshots/ folder
4. Persistence evidence (before / after)	Section 7
5. Networking, volumes and troubleshooting explanation	Sections 6, 7, 8
6. Live demo	Lab4_Part2_Demo_AhmedSalah_202200212.mp4